DEPLOYMENT SERIES · 1 OF 6

The Git Remote

Every immutable-artifact pipeline needs one thing first: a single, addressable history of what the code was at each point. That's what turns a folder into a repo.

---

concept doc — no execution in this step 2026-08-16

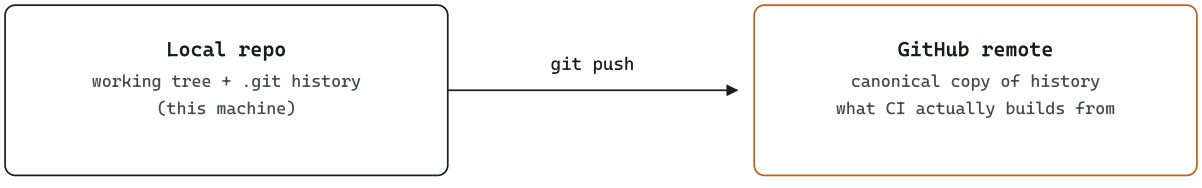
01 A commit is the same idea as an image

A git commit is a content-addressed, immutable snapshot — once made, it never changes; a new change is always a *new* commit, identified by its own hash. A container image works identically: build it, and that build gets its own address (a tag); running it later pulls the exact same bytes, never a "current" moving target.

This isn't a coincidence — it's the same idea applied twice. The pipeline this series builds turns one into the other: a commit's hash becomes an image's tag.

02 Local history vs. the remote

`git init` creates history that exists only on this machine. A **remote** is a copy of that same history hosted somewhere else — GitHub, here — that `git push` updates and everything downstream (the CI runner) reads from.



Nothing downstream — the build workflow, the runner, the registry — ever reads from this laptop directly. It only ever sees what's been pushed.

03 What actually goes in it

Not everything in the folder belongs in history. Secrets and machine-specific state stay local, same principle as the deploy directory's `.env` already documented in the main plan.

> Committed: Dockerfiles, compose files, workflow YAML, application source.
> Never committed (`.gitignore`): `.env`, credentials, anything that differs per machine.

04 The steps

- Create an empty private repository on GitHub — nothing to push to yet without it.
- `git init` in `ManPage/`
- Commit the current working tree
- `git remote add origin <repo-url>`
- `git push -u origin main`
